# fn make_select_private_candidate

Michael Shoemate

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `make_select_private_candidate` in `mod.rs`.

`make_select_private_candidate` constructs a measurement that samples a repetition count, invokes a private candidate measurement repeatedly, and either returns the first score at least a threshold or the highest-scoring sampled candidate.

## 1 Hoare Triple

### Precondition

#### Compiler-verified

- Generic `DI` implements `Domain`.
- Generic `MI` implements `Metric`.
- Generic `MO` implements `PrivateCandidateMeasure`.
- `(DI, MI)` implements `MetricSpace`.
- `measurement` has type `Measurement<DI, MI, MO, (f64, TO)>`.

#### Caller-verified

- `measurement` is a valid measurement.

### Pseudocode

```python
def make_select_private_candidate(
    measurement: Measurement,
    mean: float,
    threshold: Optional[float],
    repetitions,
) -> Measurement:
    if not mean.is_finite():  #
        raise "mean must be finite"
    if threshold is not None and not threshold.is_finite():  #
        raise "threshold must be finite"

    resolved = repetitions.resolve(mean)  #
    MO.validate(threshold is not None, resolved)  #
    function = measurement.function
    privacy_map = MO.new_privacy_map(  #
        measurement.privacy_map, threshold is not None, mean, resolved
    )
```

```
    def function_(arg):
        remaining = resolved.sample()  #
        best = None
        while remaining > 0:  #
            next_ = function(arg)  #
            if threshold is not None:
                if next_[0] >= threshold:  #
                    return (next_[0], next_[1])
            else:
                best = choose(best, next_)  #
            remaining -= 1
        return best

    return Measurement(
        input_domain=measurement.input_domain,
        input_metric=measurement.input_metric,
        output_measure=measurement.output_measure,
        function=function_,
        privacy_map=privacy_map,
    )
```

### Postcondition

**Theorem 1.1.** For every setting of the input parameters `(measurement, mean, threshold, repetitions, DI, MI, MO, TO)` to `make_select_private_candidate` such that the given preconditions hold, `make_select_private_candi` raises an error (at compile time or run time) or returns a valid measurement. A valid measurement has the following properties:

1. (Data-independent runtime errors). For every pair of members $x$ and $x'$ in `input_domain`, `invoke`$(x)$ and `invoke`$(x')$ either both return the same error or neither return an error.

2. (Privacy guarantee). For every pair of members $x$ and $x'$ in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_measure`, if $x, x'$ are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an error, and `privacy_map`(`d_in`) = `d_out`, then `function`$(x)$, `function`$(x')$ are `d_out`-close under `output_measure`.

## 2 Proof

Assume the preconditions.

**Lemma 2.1.** If the constructor returns `Ok(out)`, then:

- `resolved` satisfies the proof definition of `ResolvedRepetitions`;
- the parameter combination is accepted by `MO::validate`;
- `privacy_map` is the privacy map returned by the corresponding `PrivateCandidateMeasure` implementation.

*Proof.* Lines 7 and 9 enforce finiteness of the scalar arguments. Line 12 invokes `Repetitions::resolve`; by its postcondition, a successful return produces a valid `resolved`. Line 13 invokes the relevant `MO::validate`, so any unsupported combination is rejected before construction succeeds. Finally, line 15 invokes the relevant `MO::new_privacy_map`. □

**Lemma 2.2.** If the constructor succeeds, the returned function implements the algorithm stated in the proof definition of `make_select_private_candidate`.

*Proof.* By Lemma 2.1, line 20 samples from a valid `ResolvedRepetitions`. By the postcondition of `ResolvedRepetitions::sample`, `remaining` is an exact draw from the selected repetition law.

The loop on line 22 invokes the base `measurement` exactly once per remaining repetition. If `threshold` is present, line 25 returns immediately on the first sampled candidate whose score is at least the threshold, and otherwise the function returns `None` after the budget is exhausted.

If `threshold` is absent, line 28 updates `best`. By the postcondition of `choose`, `best` is always the highest-scoring non-NaN sampled candidate seen so far, with ties broken in favor of the most recent candidate. Therefore after the budget is exhausted, the function returns exactly the best sampled candidate, or `None` if none was retained. □

*Proof of data-independent runtime errors.* Assume the constructor succeeds, and consider any two inputs `x`, `x'` in `input_domain`.

Inside the returned function, line 20 samples from `resolved`. By the postcondition of `ResolvedRepetitions::sample`, any runtime error here comes only from the underlying sampler or numeric casts, and does not depend on `x`.

The only later fallible operation is line 23, which invokes the underlying `measurement`. By the postcondition of a valid measurement, these runtime errors are data-independent in `x`.

The remaining lines are comparisons, calls to `choose`, and loop control. The proof of `choose` shows it is infallible. Therefore the returned function has only data-independent runtime errors. □

*Proof of privacy guarantee.* By Lemma 2.1, the accepted parameter combination is one supported by the relevant `PrivateCandidateMeasure` implementation, and by Lemma 2.2 the returned function is the repeated-selection algorithm claimed by the proof definition.

If `MO = MaxDivergence`, the postcondition of `MaxDivergence` as a `PrivateCandidateMeasure` shows that the privacy map returned on line 15 is exactly the implemented pure-DP bound for the accepted branch.

If `MO = RenyiDivergence`, the postcondition of `RenyiDivergence` as a `PrivateCandidateMeasure` shows that the privacy map returned on line 15 is exactly the implemented Rényi-DP bound for the accepted branch.

The returned measurement reuses the input domain, input metric, and output measure of `measurement`, and `Measurement::new` is invoked with the function and privacy map just analyzed. Therefore, for every `d_in` where the privacy map evaluates to `d_out`, invoking the returned measurement on `d_in`-close datasets is `d_out`-close under the output measure. □

*Proof of Theorem 1.1.* The data-independent runtime error proof and privacy proof establish the two clauses of For every setting of the input parameters `(measurement, mean, threshold, repetitions, DI, MI, MO, TO)` to `make_select_private_candidate` such that the given preconditions hold, `make_select_private_candidate` raises an error (at compile time or run time) or returns a valid measurement. A valid measurement has the following properties:

1. (Data-independent runtime errors). For every pair of members $x$ and $x'$ in `input_domain`, `invoke`$(x)$ and `invoke`$(x')$ either both return the same error or neither return an error.

2. (Privacy guarantee). For every pair of members $x$ and $x'$ in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_measure`, if $x, x'$ are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an error, and `privacy_map`(`d_in`) = `d_out`, then `function`$(x)$, `function`$(x')$ are `d_out`-close under `output_measure`.

. □